VERIQO Evidence Model

The canonical description of what VERIQO means by "a piece of evidence." The authoritative implementation is `pkg/commercial/evidencefabric` (the projection) over `pkg/evidence/manifest` (the frozen state machine).

---

1. What an evidence item is

An evidence item is **not the document**. It is a durable, hash-bound record *about* a document that lives in the customer's own storage:

> *This artifact, with this SHA-256, was acquired by this collector from this source at this logical time, passed through these custody events, reached this state, and has this manifest hash — which anyone can recompute.*

VERIQO never holds the bytes. That is a deliberate architectural choice with real consequences: VERIQO's guarantee is about **integrity and lineage**, never about availability of the content itself.

---

2. The eight facets

Every `EvidenceRecord` projects exactly eight facets. Each answers a distinct question a challenger might ask.

Facet	Answers	Key fields
Identity	Which item is this, whose is it, which version?	`evidence_id`, `case_id`, `tenant_id`, `version`
Provenance	How did it get here, and has it been transformed?	`acquisition_record`, `transformation_count`, `custody_chain_head`
Integrity	Is it unaltered, and can I check that myself?	`sha256`, `sha512`, `hash_status`, `signature_status`, `manifest_hash`, `state`, `verified`
Custody	Who touched it, when, and why?	ordered `[]CustodyStep`
Source	Where did it originate?	`method`, `collector`, `origin`, `system`
Timing	When was it acquired, received, finalized?	`acquired_at`, `received_at`, `finalized_at` (logical ticks)
Trust	How is it classified and constrained?	`classification`, `markings`, `legal_hold`
Domain	What does it mean in its business context?	`maritime` / `insurance` / `trade` metadata

Plus an optional **Signature** (see §6).

`verified` is re-derived, not stored. Each time a record is projected, the manifest hash is recomputed and the custody chain re-walked. A record cannot report `verified: true` on the strength of a flag someone set once.

---

3. The lifecycle

An item advances through a real state machine. States are reached by satisfying substantive prerequisites, not by assignment:

```
SOURCE → ACQUIRE → PRESERVE → HASH → PROVENANCE → MANIFEST → CUSTODY

DRAFT
  → INGESTED
  → INTEGRITY_ASSESSED
  → PROVENANCE_COMPLETE
  → READY_FOR_FINALIZATION
  → FINALIZED          (immutable; custody chain head frozen)
      ↘ SUPERSEDED     (immutable lineage to the replacement)
```

Two properties matter commercially:

1. **Only FINALIZED evidence can ground a decision.** A decision citing a non-finalized item is refused. There is no override flag.
2. **FINALIZED is immutable.** The custody chain head freezes at finalization. Correcting an item means submitting a new version and marking the old one SUPERSEDED — which preserves the lineage rather than erasing the mistake.

---

4. Chain of custody

Custody events are hash-linked in order: each binds the previous chain head, so removing or reordering an event is detectable. Every event carries `event_id`, `actor`, `tick`, `action`, and an optional `reason`.

The Commercial API records these automatically during submission (`RECEIVED` → `HASHED` → `REVIEWED`), and additionally on legal-hold placement and release, so a hold is itself part of the custodial record rather than metadata sitting beside it.

`GET /v1/evidence/{id}/custody` returns the full chain. `POST /v1/evidence/{id}/verify` re-walks it live.

---

5. Three evidence classes

Domain metadata is a discriminated set — normally exactly one is populated:

- **Maritime** — `vessel_identity`, `port_code`, `event_kind` (e.g. `AIS_STATUS`, `PORT_EVENT`), `location`.
- **Insurance** — `claim_id`, `policy_id`, `party_id`, `evidence_kind` (e.g. `SURVEY`, `ADJUSTER_REPORT`, `FNOL`, `INVOICE`).
- **Trade** — `document_type` (e.g. `EBL`, `BILL_OF_LADING`), `transfer_event_id`, `holder_identity`.

Domain metadata is descriptive context, not an authority claim. VERIQO does not validate that a vessel identity is real or that a policy exists — it records what was asserted, by whom, at what time. Verifying an external identity against an authoritative registry is a separate integration that is **not built** (see the Real-World Network Model document).

---

6. Integrity, signing, and what each proves

Three distinct mechanisms, often conflated:

Mechanism	Proves	Present
`sha256` of the bytes	The document you hold today is the one submitted.	Always (caller-supplied).
Manifest hash	The *record about* the document — every field above — is unaltered. Computed over JCS-canonicalized content.	Always, re-derived.
Signature	A specific key attested to that manifest hash at a specific logical time.	Only when signing is enabled.

The signature, when present, is real Ed25519 over the manifest hash, carrying `key_id`, `key_version`, and `signed_at_tick`. Key revocation is retroactive: once a key is revoked, signatures it made while active fail verification.

Absence of a signature means unsigned. The field is omitted, not faked. A reviewer must not read an unsigned record as an authenticated one.

Note a deliberate honesty artifact: the frozen manifest package has its own `signature_status` / signature fields that this projection leaves unset, because populating them would require mutating a frozen package. The Commercial layer's signature is carried in the separate `signature` object instead. This is a named gap, not a silent workaround.

---

7. Retention, legal hold, and purge

Every submitted item enters retention governance at finalization — required, not best-effort, so an item cannot be finalized while ungoverned.

```
ACTIVE → RETENTION_ELIGIBLE → PURGE_ELIGIBLE → PURGED
   ↓
 HELD  ──(no edge to PURGE_ELIGIBLE)──✗
   ↓
REDACTION_REQUIRED → REDACTED
```

Legal hold blocks purge twice over: the state graph has no edge from `HELD` to `PURGE_ELIGIBLE`, and the purge operation independently refuses while any hold is present. Purging preserves the audit record of what existed and that it was purged — the ledger does not develop a hole.

Current limitation: these operations are driven through the Go API; there is no HTTP route for them yet.

---

8. What the model does not claim

- It does not claim a document is **authentic** — only that it is unaltered since submission and that its stated provenance is recorded. Whether a surveyor's report is honest is outside what any hash can establish.
- It does not claim an asserted identity (vessel, party, policy) has been **checked against an authoritative registry**.
- It does not take a **legal position** on admissibility or evidential weight. See the Legal Positioning Statement.
- `transformation_count` records that transformations occurred; it does not itself prove a transformation was faithful.